CS405 Project 1: Technical Report

Painting: Feyhaman Duran, *Kabataş Rıhtımı* (1936), Sakıp Sabancı Museum (kabatas, 1280×1026 px, aspect ≈ 1.2476).

I chose this painting for two reasons. First, it has a clear depth cue: the diagonal quay wall, the bollards and the waterline give a ground plane in front of the canvas ($z > 0$). Second, it has a sailboat. Its hull, mast, boom and gaff fit a scene graph well, and its outline is a good shape to trace with curves.

1. Part A: Transformations and Multiplication Order

Matrices are stored as `Float32Array(16)` in column-major order (index = col·4 + row) and points are column vectors, so $p' = M\,p$. For a rigid transform I use the closed-form inverse $M^{-1} = [\ R^{\mathsf{T}}\ ,\ -R^{\mathsf{T}}\mathbf{t}\ ;\ 0^{\mathsf{T}}\ ,\ 1\ ]$ (`invertRigid`).

Since the vector is on the right, the transforms are applied from right to left. With $T{\cdot}R{\cdot}S$ the cube is first scaled and rotated around its own centre and then moved to **t**, so column 4 is always **t**. With $S{\cdot}R{\cdot}T$ the cube is moved first and then rotated around the world origin, so it swings on a circle of radius $\|\mathbf{t}\|$. The scale comes last, so it also scales the translation and, because it is non-uniform, shears the rotated box. With my bollard defaults **t** = (−0.88, −0.58, 0.24), **r** = (0.20, 0.45, 0.15) rad and **s** = (0.12, 0.26, 0.12) I get:

$$M_{\text{TRS}} = \begin{vmatrix} 0.107 & -0.016 & 0.054 & -0.880 \\ 0.016 & 0.255 & -0.016 & -0.580 \\ -0.052 & 0.047 & 0.106 & 0.240 \\ 0 & 0 & 0 & 1 \end{vmatrix} \quad M_{\text{SRT}} = \begin{vmatrix} 0.107 & -0.007 & 0.054 & -0.077 \\ 0.035 & 0.255 & -0.035 & -0.187 \\ -0.052 & 0.021 & 0.106 & 0.059 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

The centre of the bollard moves from (−0.88, −0.58, 0.24) to (−0.077, −0.187, 0.059). In the rest of the project I use $T{\cdot}R{\cdot}S$, because it keeps the object where I put it and the scale stays in the object's own axes. The scene-graph nodes only use $T{\cdot}R$ and the scale is applied at the leaves, so the children never get sheared.

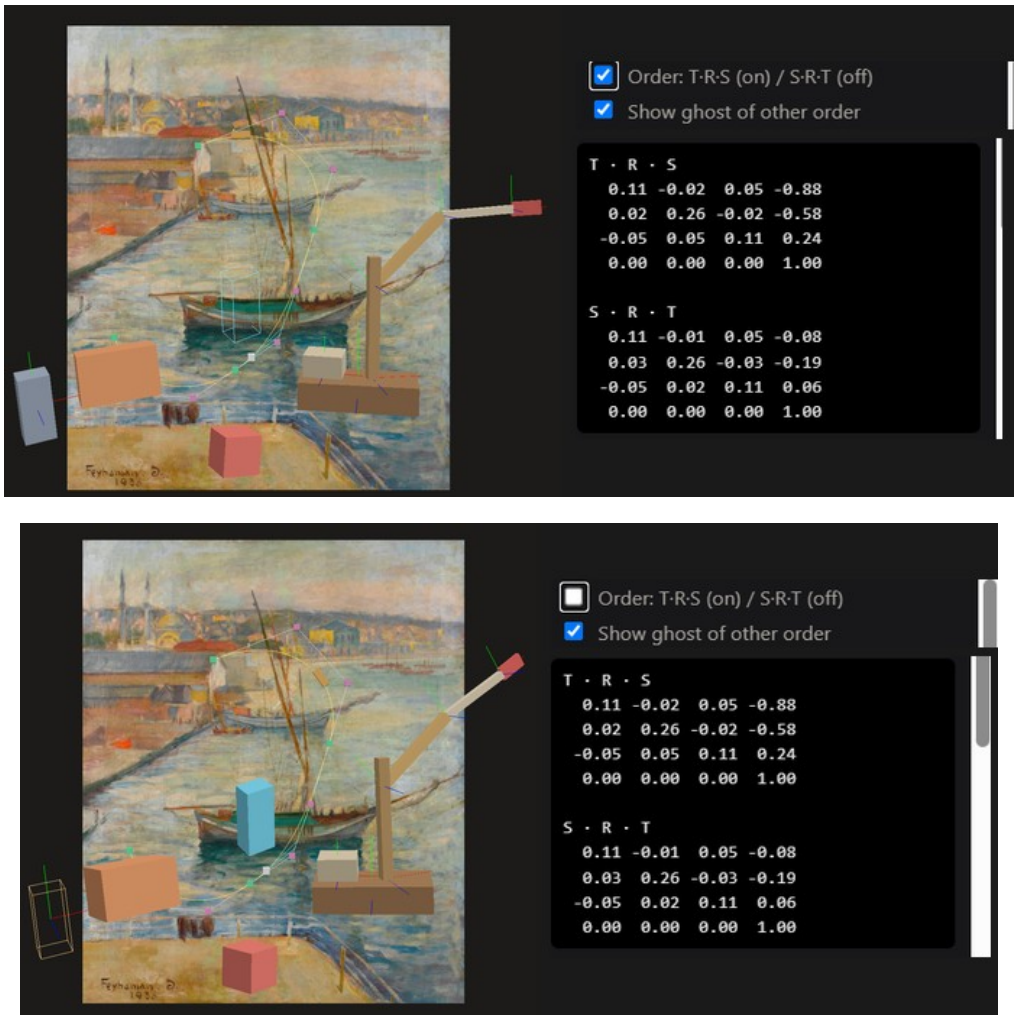


Figure 1: Part A screenshots with both matrices visible in the panel. Top: order T·R·S (checkbox on). Bottom: order S·R·T (checkbox off). The panel lists the T·R·S and S·R·T matrices for the same slider values, and the ghost (wireframe) of the other order is drawn next to the solid cube.

2. Part B: Scene Graph

The sailboat is a scene graph with 6 nodes in 5 levels. Every local matrix is rigid ($T \cdot R$) and the box sizes (`meshScale`) are only applied when drawing.

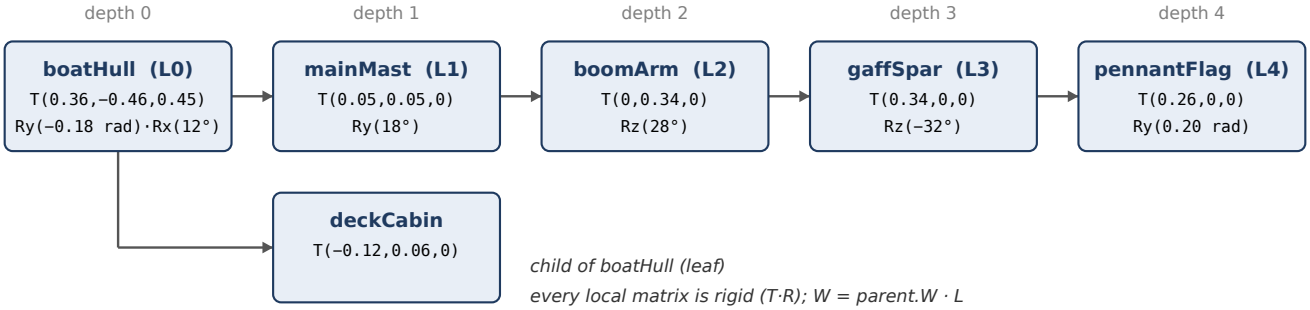


Figure 2a: scene-graph diagram of the sailboat (6 nodes, 5 levels). Each box shows the node's local matrix L as a product of rigid transforms; the box size (meshScale) is applied only when drawing.

I checked the leaf world matrix $W_4 = L_0 L_1 L_2 L_3 L_4$ by hand with the animation paused. Inside the mast plane the elbow is at (0.34 cos 28°, 0.34 + 0.34 sin 28°) = (0.3002, 0.4996) and the total Z-rotation is 28° − 32° = −4°, so the flag origin is at (0.3002 + 0.26 cos 4°, 0.4996 − 0.26 sin 4°) = (0.5596, 0.4815, 0). Then L_1 (Ry 18° plus the offset) gives (0.5822, 0.5315, −0.1729) in the hull frame, and L_0 gives the world matrix below. It matches `pennantFlag.world` from `SceneNode.updateWorld()`:

$$W_4 = \begin{vmatrix} 0.9429 & 0.0319 & 0.3314 & 0.9433 \\ 0.0352 & 0.9802 & -0.1946 & 0.0958 \\ -0.3311 & 0.1952 & 0.9232 & 0.4965 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$



Figure 2b: the boat in motion with the debug axes on (each node's local x, y, z axes drawn in red, green, blue).

3. Part C: Camera and Projection

lookAt derivation. Let **e** be the eye and **c** the target. The camera looks down its own $-z$ axis, so its backward axis is $\mathbf{w} = (\mathbf{e}-\mathbf{c})/\|\mathbf{e}-\mathbf{c}\|$. The right axis must be perpendicular to **w** and to the world up vector: $\mathbf{u} = (\mathbf{up}\times\mathbf{w})/\|\mathbf{up}\times\mathbf{w}\|$ (first cross product). The true up axis completes the orthonormal frame, $\mathbf{v} = \mathbf{w}\times\mathbf{u}$ (second cross product; it is already unit length because $\mathbf{w} \perp \mathbf{u}$). The pose of the camera in the world is the rigid matrix $C = [\ R \mid \mathbf{e}\ ]$ with $R = [\mathbf{u}\ \mathbf{v}\ \mathbf{w}]$ as columns. The view matrix maps world points into that frame, so it is the rigid inverse $V = C^{-1} = [\ R^{\mathsf{T}}\ ,\ -R^{\mathsf{T}}\mathbf{e}\ ;\ 0^{\mathsf{T}}\ ,\ 1\ ]$. The rows of R^{T} are **u**, **v**, **w**, so

$$V = \begin{vmatrix} u_x & u_y & u_z & -u\cdot e \\ v_x & v_y & v_z & -v\cdot e \\ w_x & w_y & w_z & -w\cdot e \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Check: $V\mathbf{e} = \mathbf{0}$ (the eye is the origin of view space) and $V(\mathbf{e}-\mathbf{w}) = (0, 0, -1)$, so a point in front of the camera gets a negative z, which is what the projection matrices below expect. **Degenerate case:** if $\|\mathbf{up}\times\mathbf{w}\| < 10^{-6}$ (the up vector is parallel to the view direction) **u** would be 0/0, so I switch to a fallback up vector and the camera does not flip.

Perspective and orthographic (WebGPU, z ∈ [0, 1]). With $z_c = A\, z_e + B$ and $w_c = -z_e$, the conditions $z_{\text{ndc}}(-n) = 0$ and $z_{\text{ndc}}(-f) = 1$ give $B = A\, n$ and $A = f/(n-f)$, $B = n\, f/(n-f)$:

$$P_{\text{persp}} = \begin{vmatrix} 1/(a\cdot\tan(\theta/2)) & 0 & 0 & 0 \\ 0 & 1/\tan(\theta/2) & 0 & 0 \\ 0 & 0 & f/(n-f) & nf/(n-f) \\ 0 & 0 & -1 & 0 \end{vmatrix} \qquad P_{\text{ortho}} = \begin{vmatrix} 2/(r-l) & 0 & 0 & -(r+l)/(r-l) \\ 0 & 2/(t-b) & 0 & -(t+b)/(t-b) \\ 0 & 0 & 1/(n-f) & n/(n-f) \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Near and far. Depth is hyperbolic, $z_{\text{ndc}}(d) = f/(f-n)\cdot(1 - n/d)$. When $f \gg n$, $z_{\text{ndc}}(10n) \approx 0.9$ already, so 90% of the `depth24plus` range is used up before $10n$. A tiny near therefore causes z-fighting between the painting at $z = 0$ and the curves on it. I chose $n = 0.2$ and $f = 25$, which is enough for the whole scene.



Figure 3a: the same scene under perspective (left) and orthographic (right) projection. Main camera: eye (0, 0, 2.80), target (0, 0, 0), fovY 45°, near 0.2, far 25. Under orthographic projection the cubes keep their size with depth, so there is no foreshortening.

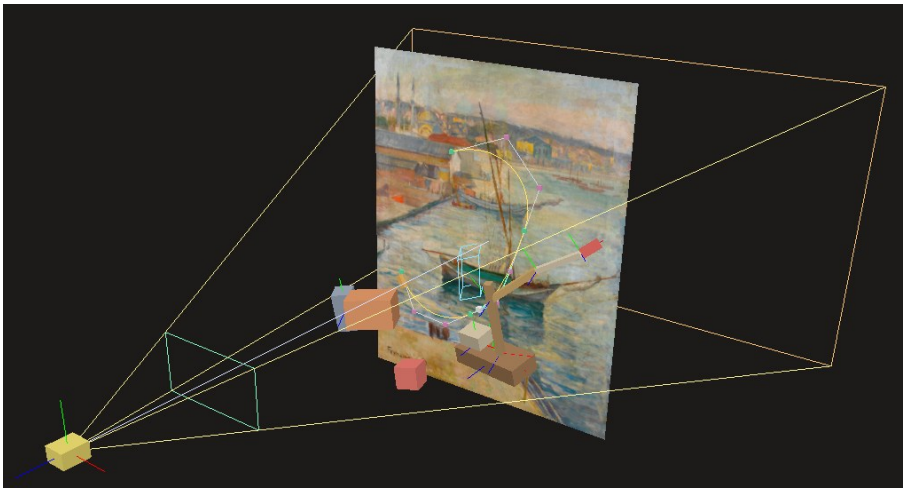


Figure 3b: the fixed inspection camera (eye (2.92, 2.02, 4.54), target (0, −0.10, 0.80)) draws the main camera's frustum; the yellow box is the main camera.

4. Part D: Curves over the Painting

Basis. $\mathbf{B}(t) = \Sigma\ B^3_i(t)\mathbf{P}_i$ with $B^3_0 = (1-t)^3$, $B^3_1 = 3(1-t)^2 t$, $B^3_2 = 3(1-t)t^2$ and $B^3_3 = t^3$. They are all non-negative and they sum to $((1-t) + t)^3 = 1$.

Endpoints. At $t = 0$ the weights are (1, 0, 0, 0), so $\mathbf{B}(0) = \mathbf{P}_0$. At $t = 1$ they are (0, 0, 0, 1), so $\mathbf{B}(1) = \mathbf{P}_3$. ■

C^1 at a joint. The derivatives at the ends are $\mathbf{B}'(0) = 3(\mathbf{P}_1 - \mathbf{P}_0)$ and $\mathbf{B}'(1) = 3(\mathbf{P}_3 - \mathbf{P}_2)$. For the next segment $\mathbf{Q}$ with $\mathbf{Q}_0 = \mathbf{P}_3$, equal derivatives give

$$3(\mathbf{P}_3 - \mathbf{P}_2) = 3(\mathbf{Q}_1 - \mathbf{P}_3) \iff \mathbf{Q}_1 = 2\mathbf{P}_3 - \mathbf{P}_2,$$

so the joint is the midpoint of its two handles. When dragging, a knot moves both of its handles, and a handle mirrors the opposite one (`c1NextControl`).

My curve has 3 segments (10 points) and traces the boat. The second curve type is a uniform Catmull–Rom spline. The cube that moves along the curve is rotated using the columns $\mathbf{T} = \mathbf{P}'/|\mathbf{P}'|$ and $\mathbf{N} = (-T_y, T_x)$. Points are picked by intersecting the mouse ray with $z = 0$. The exported JSON (normalised u, v) is given in Appendix A.



Figure 4: traced piecewise cubic Bézier curve over the boat hull, with the control polygon (knots in green, handles in pink) and the cube animated along the curve.

5. Part E: Performance

Windows 11, Chromium WebGPU, 1280×800, `depth24plus`. **GPU:** NVIDIA T550. “Frame time” below is the CPU time per frame from the app’s readout; the displayed frame interval stays at about 16.7 ms (60 fps, vsync-limited).

Configuration	Draw calls	Vertices / frame	Frame time
Main camera (axes + Bézier + de Casteljau)	28	~1,042	0.4–0.9 ms
Inspection camera (frustum on)	30	~1,108	0.5–1.0 ms

6. What Went Wrong

1. **All cubes were drawn at the last position.** I used one uniform buffer and called `queue.writeBuffer` before every draw. The writes all happen before the render pass actually runs, so every draw saw the last value. I fixed it with a pool of uniform buffers (`#acquireSlot`).
2. **Black canvas when drawing lines.** `LineRenderer` expects 6 floats per vertex (xyz + rgb), but my helper functions pushed 7 (rgba). Removing the alpha fixed the stride.
3. **`p0.map is not a function`.** `evalPiecewise` wants an array of 4-point segments, but I was passing the flat array of 10 points. `getBezierSegments()` now splits it into [0..3], [3..6] and [6..9].
4. **Roll was actually pitch.** The long axis of the hull is +X, so `rotateZ` tipped the bow up and down instead of rolling the boat. I changed the roll to `rotateX`.

7. Bonus

1. **Least-squares fit.** In trace mode the stroke is split into 3 parts of equal arc length. For each part I use chord-length parameters t_k, fix the endpoints, and get $\mathbf{P}_1$, $\mathbf{P}_2$ from the 2×2 normal equations:

$$\begin{vmatrix} \Sigma\, B1^2 & \Sigma\, B1B2 \\ \Sigma\, B1B2 & \Sigma\, B2^2 \end{vmatrix} \begin{vmatrix} P1 \\ P2 \end{vmatrix} = \begin{vmatrix} \Sigma\, B1 \cdot rk \\ \Sigma\, B2 \cdot rk \end{vmatrix}$$

   After that the joints are made C^1 by averaging the tangents.
2. **Surface of revolution.** The curve profile is revolved around the vertical axis $x = -0.15$ as $\mathbf{S}(u, \varphi) = (x_0 + r(u) \cos \varphi, y(u), z_0 + r(u) \sin \varphi)$, on a 40 × 20 grid. The normals come from the curve tangent, so the mesh is lit.
3. **Painter's camera.** The Bosphorus waterline gives a horizon at $y \approx 0.18$, a bit above the canvas centre, which suggests the painter stood on the raised pier. The edges of the quay wall meet at ≈ (−0.18, 0.18) on that horizon. I matched this with fovY = 38°, azimuth −11°, elevation +8° and distance 2.95 (view 4, with the horizon and rays drawn as an overlay).
4. **Quaternion orbit.** The camera orientation is a unit quaternion and the offset is rotated by $q\mathbf{v}q^*$. When the view changes I interpolate with $\text{slerp}(q_0, q_1, t) = [\sin((1-t)\Omega)q_0 + \sin(t\Omega)q_1] / \sin \Omega$, using a shortest-arc check ($q_0 \cdot q_1 < 0 \Rightarrow q_1 = -q_1$) and nlerp when Ω is small.
5. **Compute shader ribbon.** A `@workgroup_size(64)` compute pass evaluates $\mathbf{C}(u)$ and $\mathbf{C}'(u)$ at 128 samples and writes 2 triangles per sample into a `STORAGE | VERTEX` buffer. That buffer is drawn directly, without reading anything back to the CPU.



Figure 5: bonus screenshots. Left: surface of revolution of the traced curve, lit with normals from the curve tangent. Right: painter's-perspective view with the horizon (y ≈ 0.18) and the rays through the quay-wall edges drawn as an overlay.

Appendix A: Exported curve JSON

Control points in normalised painting coordinates $(u, v) \in [0, 1]^2$, exported by the app (full file in the repository: `kabatas-curve.json`). Segments: points [0..3], [3..6], [6..9].

```
[[0.2942,0.58],[0.3066,0.645],[0.4002,0.655],[0.4875,0.655],[0.5749,0.655],[0.6809,0.63],
[0.7557,0.585],[0.8306,0.54],[0.9055,0.515],[0.9741,0.495]]
```

Appendix B: AI Usage

Tool: Gemini

Asked about	What I kept
Blank render when pushing rgba vertices for lines	LineRenderer stride is 6 floats, so I push rgb only
All meshes using the last transform	writeBuffer timing, so one uniform buffer per draw (#acquireSlot)
Control points drifting when dragging in 3D	ray–plane intersection with z = 0, $t = -o_z/d_z$
Resetting the panel sliders	calling .update() on the stored controllers in reset()
Why < 1e-6 and not === 0 in lookAt	float rounding; avoids NaN near vertical views
Formatting of this report	the layout of matrices and equations